Crew Training Functional Specification Document (FSD)

Crew Training — Functional Specification Document (FSD)

Document type: As-built (current code only — not a future-state design)
Companion page: Crew Training — Product Requirements Document (PRD)
Source of truth: `CrewTraining-main` repository, primarily `Forms\\frmCrewTraining.vb`
Runtime: .NET Framework 4.8 VB Shop Windows Forms
Analysis date: 2026-09-02

> There are **no HTTP APIs**. "API contracts" in this FSD are SQL queries, stored procedures, staging tables, and in-memory DataSet shapes.
> **Confluence import:** Paste as a **separate page** from the PRD. Use the Mermaid macro for diagrams (or paste flows into Napkin AI - Turn Text into AI Diagrams and Visuals).

Inventory (for navigation)

Layer	What exists
UI	Single form `frmCrewTraining` — tabs Employees / Classes / Topics
Services (in-process)	`SQLJammer`, `FileExplorer`, `GenericPicker`, `SqlBulkCopy`, `StartFinesseProcess`
HTTP APIs	None
Database	SQL Server; see §7 and §8
AuthZ	None in this app — see User Access Matrix

Feature IDs F01–F21 match the PRD functional scope.

1. Feature-by-Feature Functional Specs

F01 — Application startup and initial load

Purpose: Open SQL, load all training data into memory, default the company filter.

Inputs

Source	Data
OS	Windows identity
ClairUtilities	`FinesseConnectionString`
SQL	Queries 1–11 in §8.1

Command line	If `CommandLineArgs.Count > 0`, explicitly set `MainForm = New frmCrewTraining()` (already the default)

Processing

1. `MyApplication_Startup`: `gConn.Open()`.
2. Main form = `frmCrewTraining`.
3. `Form_Load`: `KeyPreview = True`; bind DataTables from `_dsCrewTrainingData`.
4. `SQLJammer(GetOpenedFinesseConnection).Execute()` runs all SELECTs.
5. Fill `cboCompanyFilter` with `"All Companies"` + each `CompanyDesc`.
6. Select company matching `_dtUserInfo.Rows(0).CompanyCode`.
7. That selection fires `populateLvEmployees()` (active employees for that company).
8. Fix picker column display indexes; `populatTVCMS()`; collapse add-topic panel; attach list sorters; enable double buffering.
9. `_isLoaded = True`.

Outputs / side-effects

- In-memory DataSet filled.
- Trainers combo bound: `DisplayMember=fullname`, `ValueMember=user_name`.
- No DB writes.

Errors

- SQL/connection failures: unhandled in `Form_Load` (crash on startup).
- Empty `_dtUserInfo`: `IndexOutOfRange` on `Rows(0)`.
- Empty `_dtCompanies` match: `r(0)` fails.

Access: any user who can open the SQL connection.

F02 — Employee search, company filter, inactive filter

Purpose: Find employees to attach training to.

Inputs (UI)

Control	Meaning
`txtEmployeeSearch`	Tokens split on space; match `firstname`, `lastname`, or `empno` with `LIKE %term%`

`cboCompanyFilter`	`"All Companies"` or a `CompanyDesc`
Include/Exclude Inactive menu	Sets `_IncludeInactiveEmployees` (default Exclude checked)

Processing (`populateLvEmployees`) — as coded, including BUG-01

| Search box | Include inactive flag | Filter actually applied |
|---|---|---|
| Empty | False (default Exclude) | `empstatus = 'A'` (+ company if not All) |
| Empty | True | All employees (no status filter) |
| Non-empty | True (Include) | `AND empstatus = 'A'` — opposite of the menu label |
| Non-empty | False (Exclude) | No empstatus predicate — inactive employees can appear |
Assumption: This inversion is a defect, not intended business policy.
Debounce: 1000 ms `Threading.Timer` → `Invoke(populateLvEmployees)`.
Output: `lvEmployees` columns empno, firstname, lastname. Default sort column 2 (lastname) ascending via `ListViewColumnSorter`.

F03 / F04 / F05 — Employee training topics grid

Purpose: Maintain `peEmployeeTrainingTopics` per employee.

Inputs

- Selected `lvEmployees` item text = `empno`.
- Filter: `bsEmployeeTrainingTopics.Filter = "empno = " & SQLQuote(empno)`.
- Grid columns: topic (RO), training start, training end, classname (RO), instructor (display = trainer fullname), two picker buttons.

Add row (`tsbAddTrainingRow`)

1. Require selected employee else `"Please Select an Employee."`
2. `pickTopic()` GenericPicker on `_dtTrainingTopics` (includes inactive topics).
3. If current training row exists, reject duplicate where same `empno` + `topic` + `trainingDateStart = bsClasses.Current("StartDate")` → `"Cannot copy this line because it already exists."`
4. `AddNew`; set `empno`, `topic`, `trainingDateStart = bsClasses.Current("StartDate")`.
5. Catch → `"Failed to add rows this probably should not happen: " & ex.Message`

Hidden dependency (BUG-02): Start date is taken from **whatever class is current in**

`bsClasses`, not from Today and not from a class the user picked on this tab. On first load `bsClasses` is unfiltered, so Current is typically the **oldest class** (`ORDER BY startdate ASC`). **Instructor and classname are not set** by this add path.

Delete: Require current row else `"Please Select a Training Topic row to delete."`; `RemoveCurrent()`.

Pickers on cell click: replace `topic` or set `instructor` to trainer `user_name`.

EndEdit errors: `"error editing row: "`.

Output: In-memory `EmployeeTraining` row Added/Deleted/Modified; persisted only on Save.

F06 — Employee classes attended

Purpose: Show `TrainingClassStudents` for the selected employee; edit notes and final grade.

Inputs / bindings

UI	Binding
`dgEmployeeClasses`	`classname` (RO), `FinalGrade` (editable, max 25)
`txtEmpClassNotes`	`Notes` (max 2000 in XSD)

Filter: `empno = selected`. TextChanged on notes calls `EndEdit()` (empty catch).

Not shown in this grid: `FirstTour`, `Termination`, `startdate` (loaded, saved if dirty, no editor).

Context menu `mnuEmployeeTrainingGridContext` Delete has no handler.

F07 — Class list and search

Purpose: Browse classes in four groups.

Group index	Header	Rule
0	Current Classes	`enddate` not null AND `startdate <= Today` AND `enddate >= Today`
1	Future Classes	`enddate` not null AND `startdate >= Today`
3	Historical Classes	`enddate` not null AND `enddate <= Today`
2	Classes Missing Dates	`enddate` is `DBNull`

Search: `classname LIKE %term%` plus multi-word AND group. Results reversed (source was ASC then `list.Reverse()`).

Edge (BUG-05): Clearing `txtClassSearch` to `""` clears the list and exits — does not reload all classes.

Classes tab first visit: if list empty, populate all (empty search = `LIKE '%%'`).

Selection filters: `bsTrainingClassStudents`, `bsTrainingClassTopics`, `bsClasses` by `classname`.

F08 — Create class

Purpose: Insert a new `TrainingClasses` row.

Processing

1. `bsClasses.AddNew`.
2. `classname = getNewClassName()` → `"New Class"` or `"New Class n"`.
3. `startdate = Today()`, `enddate = DBNull`, `instructor = DBNull`.
4. Append ListView item and select it.

   Output: In-memory insert; ListView not regrouped until Save (Save on Classes tab calls `populateLvClasses()`).

   No uniqueness check against SQL until commit. DataSet PK is `(classname, startdate)`.

F09 — Edit class header

Field	Control	Binding
Class name	`txtNewClassName`	`bsClasses.classname`
Instructor	`cboTrainers`	`SelectedValue` → `instructor` (`user_name`)
Start	`dtpClassStartDate`	`startdate`
End	`dtpClassEndDate`	`enddate`